

Cryptography and Secure Communication

Symmetric and asymmetric encryption · hash functions · steganography · password storage · data in transit and at rest · attacks and defenses

Class 14 · Cyber Defense (420-D03-AB) · **AEC** (Attestation d'études collégiales) LEA.D8 · Competency EZ03 · Wednesday, September 30, 2026 · 6:00 pm **EDT** (Eastern Daylight Time)



Cyber.SoHo Educational Hub



Before we start: sign, then practise safely

- **Sign today's Acknowledgment and Agreement page** (Legal Notice, section 16); the signing page is provided separately
- **Practise only in your isolated lab** (the LAB_01 environment) or on systems for which you hold **written permission** obtained before you start
- **Never** scan, intercept, crack or access a system, network, Wi-Fi or account that is not yours
- Password, credential and interception attacks against real accounts or real communications are **forbidden**
- Unauthorized access is a criminal offence: Criminal Code of Canada, sections **342.1**, **342.2** and **430(1.1)**
- In doubt about a system? **Stop and ask the instructor** before doing anything

1 · Sign

Session acknowledgment, section 16

2 · Isolated lab only

Snapshots and isolated virtual networks, section 3

3 · Stop and ask

Any doubt about ownership, section 4

Sources and links: <https://laws-lois.justice.gc.ca/eng/acts/C-46/section-342.1.html> · <https://laws-lois.justice.gc.ca/eng/acts/C-46/section-342.2.html> · <https://laws-lois.justice.gc.ca/eng/acts/C-46/section-430.html>



The Legal Notice: what applies to this lecture

- **Educational purpose only:** the Material teaches how to protect systems, never how to attack others (section 2)
- **Your risk, your responsibility:** equipment, accounts, cloud costs and time (5); the Material is provided **as is**, without warranty (6)
- **No professional advice:** not legal advice, not security consulting (7)
- **Intellectual property:** © 2026 Cyber.SoHo; personal study licence, no redistribution or public posting (8); no affiliation with any product, organization or channel named (9)
- **External links and videos:** check every link with the VirusTotal **URL** (Uniform Resource Locator) scanner; third-party clips and comments are not endorsed (10, 12)
- **Ethics and AI (Artificial Intelligence):** use your skills only to protect; report accidental findings responsibly; state the **LLM** (Large Language Model) and the exact prompts you used (11)
- **Indemnification** (13) · **Privacy:** these slides and pages collect no personal data (14) · **Governing law:** Québec (15)



Sources and links: <https://www.virustotal.com/gui/home/url>

Recall of Class 13, and today's outcomes

Class 13 in three facts

- **WPA3 (Wi-Fi Protected Access 3)** replaces the pre-shared-key exchange of **WPA2** (Wi-Fi Protected Access 2) with **SAE (Simultaneous Authentication of Equals)**, which resists offline guessing of the Wi-Fi passphrase
- **Guest Wi-Fi** is a separate zone that reaches the Internet only
- **Shared responsibility**: the cloud provider secures the infrastructure; the customer secures data, identities and configuration

By the end of today you can

- Distinguish symmetric encryption, asymmetric encryption, hash functions and steganography
- Explain why systems store **salted, slow hashes** and read an `/etc/shadow` record
- Name the controls for **data in transit** and **data at rest**, and how to validate them
- Recognize common **attacks on cryptosystems** and match each with a defense



Today's plan, and where Class 14 fits

Time (EDT)	Part	Duration
6:00 – 6:04	Recall and outcomes	4 min
6:04 – 6:22	Lesson 1: fundamentals	18 min
6:22 – 6:35	Lesson 2: password storage	13 min
6:35 – 6:48	Lesson 3: in transit and at rest	13 min
6:48 – 6:59	Lesson 4: attacks and defenses	11 min
7:00 – 7:30	Code demos, flashcards, questions	30 min
After the break	Guided exercise (not graded)	work time

- Classes 14 to 17 form the **cryptography block**
- Class 15 (Tuesday, October 6): **PKI (Public Key Infrastructure)** and certificates
- Classes 16 and 17: **LAB_06, VPN** (Virtual Private Network) setup and encryption, which becomes Chapter 6 of your Final Project and Course Procedure Manual
- **Quiz Q_03 is moved to Class 16** (Wednesday, October 7)



1

LESSON 1 OF 4 · ABOUT 18 MINUTES

Cryptography fundamentals

Symmetric encryption, asymmetric encryption, hash functions and steganography

- 1.1 What cryptography does, and the vocabulary
- 1.2 Symmetric encryption: one shared key
- 1.3 Asymmetric encryption: a key pair
- 1.4 Hybrid encryption
- 1.5 Hash functions
- 1.6 A brief look at steganography



What cryptography gives: four security services

- **Cryptography**: protecting information with mathematical algorithms, so that only authorized parties can read it and changes or a false origin can be detected

Confidentiality

Only authorized parties can read the information

Integrity

Any change, accidental or deliberate, is detected

Authentication

You can verify who produced the data or who is at the other end

Non-repudiation

A signer cannot credibly deny a signature made with the private key

- Not provided by cryptography: **availability** (redundancy, backups, **DoS** (Denial of Service) defenses, Class 12)

Core vocabulary, key space and Kerckhoffs's principle

- **Plaintext**: readable data · **Ciphertext**: the unreadable result
- **Encryption** turns plaintext into ciphertext; **decryption** reverses it
- **Algorithm** (**cipher**): the published procedure · **Key**: the secret value that makes the result unique
- **Key length**: size of the key in bits · **Key space**: all possible keys = 2^{length}
- **Kerckhoffs's principle** (1883): a system must stay secure even when everything except the key is public
 - Use published, standardized algorithms (**AES** (Advanced Encryption Standard), **SHA-256** (Secure Hash Algorithm, 256-bit output), **RSA** (Rivest–Shamir–Adleman), **ECC** (Elliptic Curve Cryptography)), never a home-made method
 - Canada: **CCCS** (Canadian Centre for Cyber Security) ITSP.40.111 lists approved algorithms · United States: **NIST** (National Institute of Standards and Technology) **FIPS** (Federal Information Processing Standards) standards

$$2^{128} \approx 3.4 \times 10^{38}$$

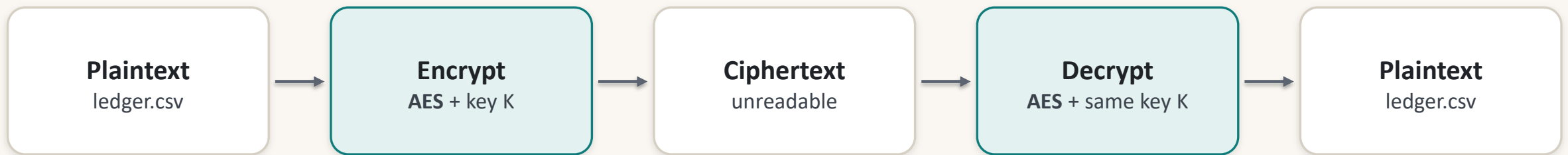
possible values of a 128-bit key

Sources and links: <https://www.cyber.gc.ca/en/guidance/cryptographic-algorithms-unclassified-protected-protected-b-information-itsp40111> · <https://csrc.nist.gov/>



Symmetric encryption (1/3): one shared key

- **Symmetric encryption**: the same secret key encrypts and decrypts: $C = E(K, P)$ and $P = D(K, C)$
- **Block cipher**: works on fixed-size blocks. **AES (Advanced Encryption Standard)**: 128-bit blocks; keys of 128, 192 or 256 bits; 10, 12 or 14 rounds (**FIPS** (Federal Information Processing Standards) 197)
- **Stream cipher**: a keystream combined with the plaintext bit by bit (**XOR** (exclusive OR)). **ChaCha20** (**RFC** (Request for Comments) 8439), used by WireGuard
- Retired: **DES (Data Encryption Standard)** (56-bit key found in 56 hours in 1998) · **Triple DES** (phased out by the end of 2023)



The same key K must reach both sides through a channel the attacker cannot read

Sources and links: <https://csrc.nist.gov/pubs/fips/197/final> · <https://www.rfc-editor.org/rfc/rfc8439> · <https://www.eff.org/press/releases/eff-des-cracker-machine-brings-honesty-crypto-debate>



Symmetric encryption (2/3): modes of operation and the nonce

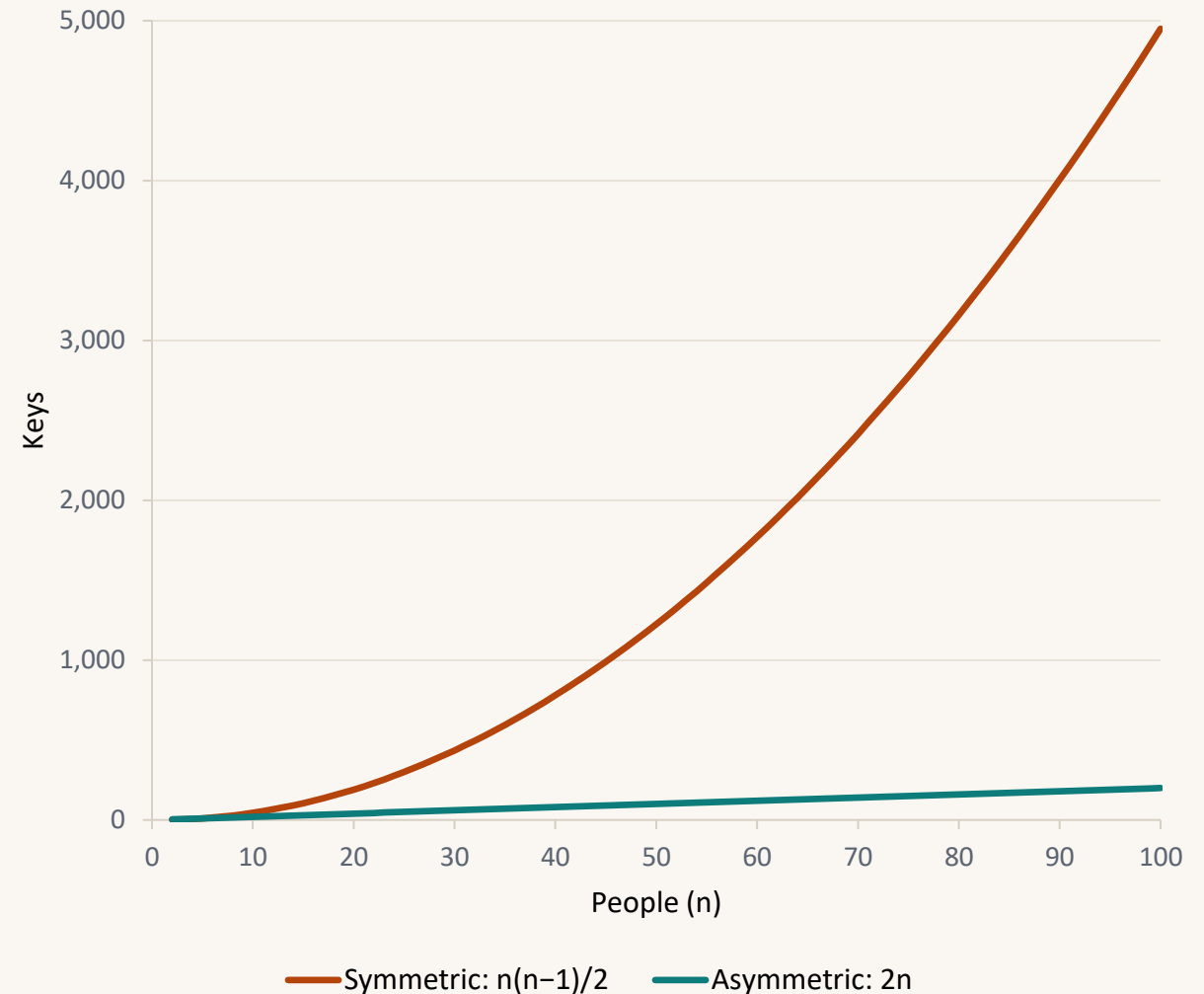
- **Mode of operation**: the rule that applies a block cipher to a whole file; it matters as much as the algorithm
- **Nonce**: a number used once (96 bits for **GCM** (Galois/Counter Mode)). Reusing it with the same key reveals plaintext information and allows forged tags
- **AEAD (Authenticated Encryption with Associated Data)**: a mode that gives confidentiality and integrity together (**GCM**, ChaCha20-Poly1305)

Mode	How it works	Verdict
ECB (Electronic Codebook)	Each block encrypted alone: identical blocks give identical ciphertext	Never use (Figure 4.1)
CBC (Cipher Block Chaining)	Each block chained to the previous one; random initialization vector	Hides patterns, no integrity
GCM	Counter mode + 128-bit authentication tag	Confidentiality + integrity
XTS (XEX-based Tweaked-codebook mode with ciphertext Stealing)	Designed for disk sectors	BitLocker, disk encryption



Symmetric encryption (3/3): the two key problems

- Strengths: very fast, short keys, ideal for bulk data (disks, **VPN** (Virtual Private Network) traffic)
- **Key distribution problem**: both sides need the same key, delivered through a channel the attacker cannot read
- **Scalability**: one key per pair of people = $n(n-1)/2$ keys
 - NorthMaple, 10 people: **45** keys
 - 100 people: **4,950** keys · 1,000 people: **499,500** keys
- Asymmetric cryptography needs only $2n$ keys (next slide)



Asymmetric encryption (1/2): a key pair

- **Asymmetric encryption (public-key cryptography)**: two mathematically linked keys
- **Public key**: given to anyone · **Private key**: kept secret, never leaves its owner
- **One-way function**: easy to compute, infeasible to reverse
- **RSA (Rivest–Shamir–Adleman)** (1977): factoring large numbers · **ECC (Elliptic Curve Cryptography)**: elliptic-curve discrete logarithm
- 256-bit **ECC** $\approx$ 3072-bit **RSA** (about 128-bit security)
- n people need **$2n$ keys** (NorthMaple: 20); only public keys are distributed
- Costs: much slower than **AES** (Advanced Encryption Standard); small data only (**RSA-3072** (Rivest–Shamir–Adleman, 3072-bit modulus) with **OAEP** (Optimal Asymmetric Encryption Padding): at most 318 bytes)
- Who really owns a public key? Certificates and **PKI** (Public Key Infrastructure), Class 15

Public key

Shared with anyone: used to encrypt for the owner and to verify the owner's signatures

one key pair

Private key

Kept by the owner only: used to decrypt and to sign



Asymmetric encryption (2/2): three different uses

1 • Confidentiality

Encrypt with the RECIPIENT'S public key; only her private key decrypts

2 • Digital signature

Sign the hash with the SIGNER'S private key; anyone verifies with the public key: integrity, authentication, non-repudiation

3 • Key agreement

Diffie–Hellman (1976); **ECDH** (Elliptic-Curve Diffie–Hellman) with X25519: both sides compute the same secret, which never travels (WireGuard, **TLS** (Transport Layer Security) 1.3)

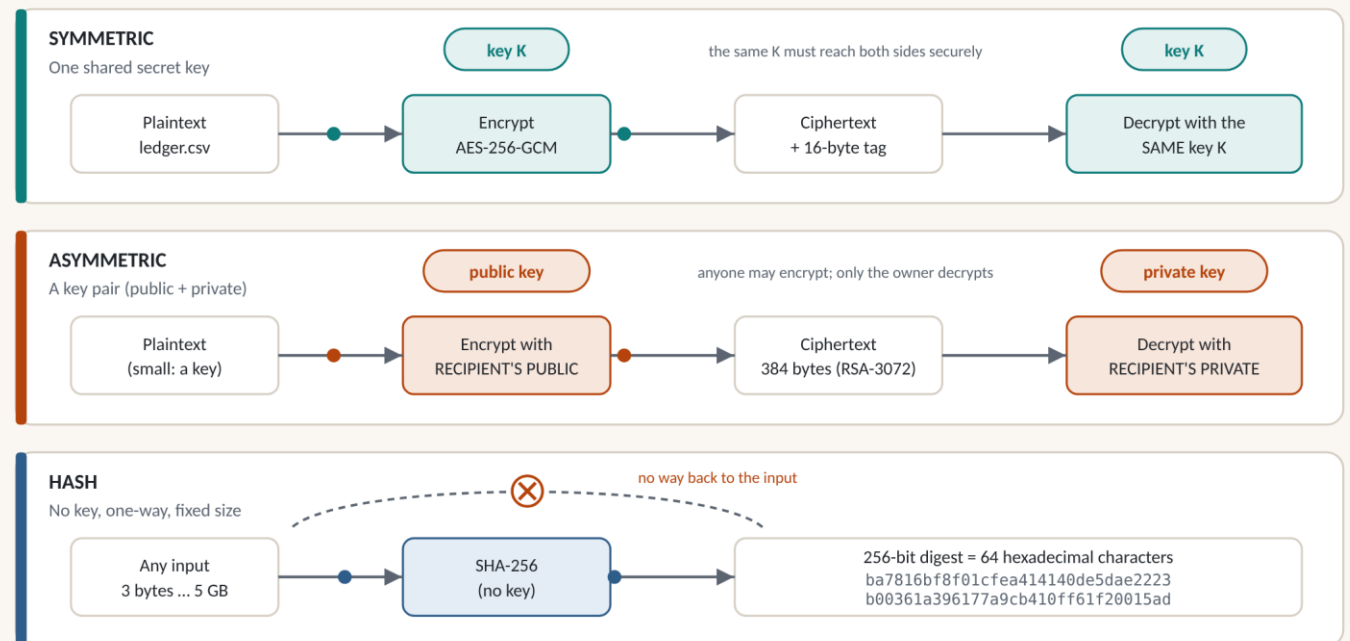
- With **RSA** (Rivest–Shamir–Adleman), what one key transforms only the other key of the pair reverses
- Elliptic-curve keys are used for key agreement (**ECDH**, X25519) and signatures (**ECDSA** (Elliptic Curve Digital Signature Algorithm), Ed25519)



Hybrid encryption: how real systems combine both

- **Hybrid encryption** combines the strengths of both families:
 - 1. Asymmetric cryptography authenticates and establishes a random **session key**
 - 2. Symmetric **AEAD** (**AES-GCM** (Advanced Encryption Standard in Galois/Counter Mode), ChaCha20-Poly1305) protects the data
 - 3. A hash or authentication tag protects integrity
- Used by **TLS** (Transport Layer Security) 1.3, **SSH** (Secure Shell) and WireGuard
- WireGuard: Curve25519 (**ECDH** (Elliptic-Curve Diffie–Hellman)), ChaCha20-Poly1305, BLAKE2s

Figure 1.4 — Symmetric encryption, asymmetric encryption and hashing, side by side



Example digest shown: SHA-256 of the three bytes "abc". Real protocols combine all three rows (hybrid encryption, section 1.4).



Hash functions (1/2): a fixed-size, one-way digest

- **Cryptographic hash function**: any input → fixed-size **digest**; no key, not reversible, not encryption
- **SHA-256** (Secure Hash Algorithm, 256-bit output): always 256 bits = 64 hexadecimal characters, for 3 bytes or for 5 **GB** (gigabyte)
- **Deterministic** · **preimage resistance** (one-way) · **second-preimage resistance** · **collision resistance**
- **Avalanche effect**: "abc" → `ba7816bf...15ad`, "abd" → `a52d159f...49c9`: about half of the bits change



256 output bits of SHA-256 — rust = bit changed (122 of 256, 47.7 %)

Figure 1.2: the 256 output bits of **SHA-256**("abc") vs **SHA-256**("abd"); rust = changed bit



Hash functions (2/2): algorithms and where defenders use them

- Broken for collisions: **MD5 (Message Digest 5)** (since 2004) and **SHA-1 (Secure Hash Algorithm 1)** (SHAttered, 2017)
- Current: **SHA-2 (Secure Hash Algorithm 2)** (**SHA-256** (Secure Hash Algorithm, 256-bit output), **SHA-384** (Secure Hash Algorithm, 384-bit output), **SHA-512** (Secure Hash Algorithm, 512-bit output); **FIPS** (Federal Information Processing Standards) 180-4) and **SHA-3 (Secure Hash Algorithm 3)** (FIPS 202)
- **Birthday bound**: a collision search needs about $2^{(n/2)}$ attempts, so **SHA-256** gives about 128-bit collision security
- Uses: file integrity · digital signatures · password storage (Lesson 2) · **HMAC (Hash-based Message Authentication Code)** (hash + secret key: integrity and origin)
- A digest proves integrity only if the reference digest is trusted (Ubuntu signs SHA256SUMS with **GPG** (GNU Privacy Guard))

VERIFY A DOWNLOAD (EXAMPLE OUTPUT FORMAT)

```
$ printf abc | sha256sum
ba7816bf8f01cfea414140de5dae2223
b00361a396177a9cb410ff61f20015ad  -

$ sha256sum -c SHA256SUMS
ubuntu-26.04.1-live-server-amd64.iso: OK
```

Sources and links: <https://shattered.io/> · <https://csrc.nist.gov/pubs/fips/180-4/upd1/final> · <https://ubuntu.com/tutorials/how-to-verify-ubuntu> · <https://www.rfc-editor.org/rfc/rfc2104>



Steganography: hiding that a message exists

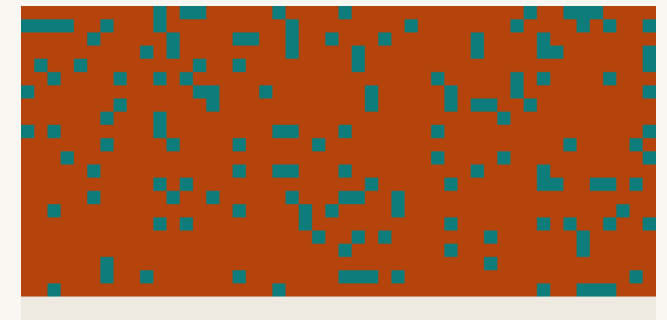
- **Steganography** hides the existence of a message; cryptography hides its meaning
- **Cover medium** + message → **stego object**; often encrypt first, then hide
- **LSB (Least Significant Bit) substitution**: change the least significant bit of each colour value (± 1 out of 255: invisible)
- Capacity: $800 \times 600 \text{ pixels} \times 3 \text{ bits} = 1,440,000 \text{ bytes}$
- Defender view: hidden malware or data exfiltration (MITRE **ATT&CK** (Adversarial Tactics, Techniques, and Common Knowledge) T1027.003); detection = **steganalysis**, **DLP** (Data Loss Prevention), **IDS** (Intrusion Detection System)
- Legitimate use: digital watermarking



Cover medium



Stego object



Zoom, 48×24 pixels: rust = changed by 1, teal = carrier unchanged

Sources and links: <https://attack.mitre.org/techniques/T1027/003/>



Videos for Lesson 1 (each 10.5 minutes or less)

LESSON 1 · FUNDAMENTALS



Public Key Infrastructure - CompTIA Security+ SY0-701 - 1.4

Professor Messer · 9:08 ·

<https://www.youtube.com/watch?v=xHAMEF7-inQ>

Symmetric encryption, its scalability limit, asymmetric key pairs and how a key pair is generated. Watch it first.



Key Exchange - CompTIA Security+ SY0-701 - 1.4

Professor Messer · 3:39 ·

<https://www.youtube.com/watch?v=U6BWn81P5Ec>

How a symmetric session key is agreed with asymmetric cryptography: the hybrid pattern of section 1.4.



Hashing and Digital Signatures - CompTIA Security+ SY0-701 - 1.4

Professor Messer · 10:24 ·

<https://www.youtube.com/watch?v=EcGmQjI6XEo>

Hash properties, **SHA-256** (Secure Hash Algorithm, 256-bit output) demonstration, collisions, **MD5** (Message Digest 5), file verification and the digital signature process.



Obfuscation - CompTIA Security+ SY0-701 - 1.4

Professor Messer · 8:00 ·

<https://www.youtube.com/watch?v=LfuTMzZke4g>

Steganography in images, audio, video and network traffic (first part of the video).



Public Key Cryptography - Computerphile

Computerphile · 6:19 ·

https://www.youtube.com/watch?v=GSIDS_lvRv4

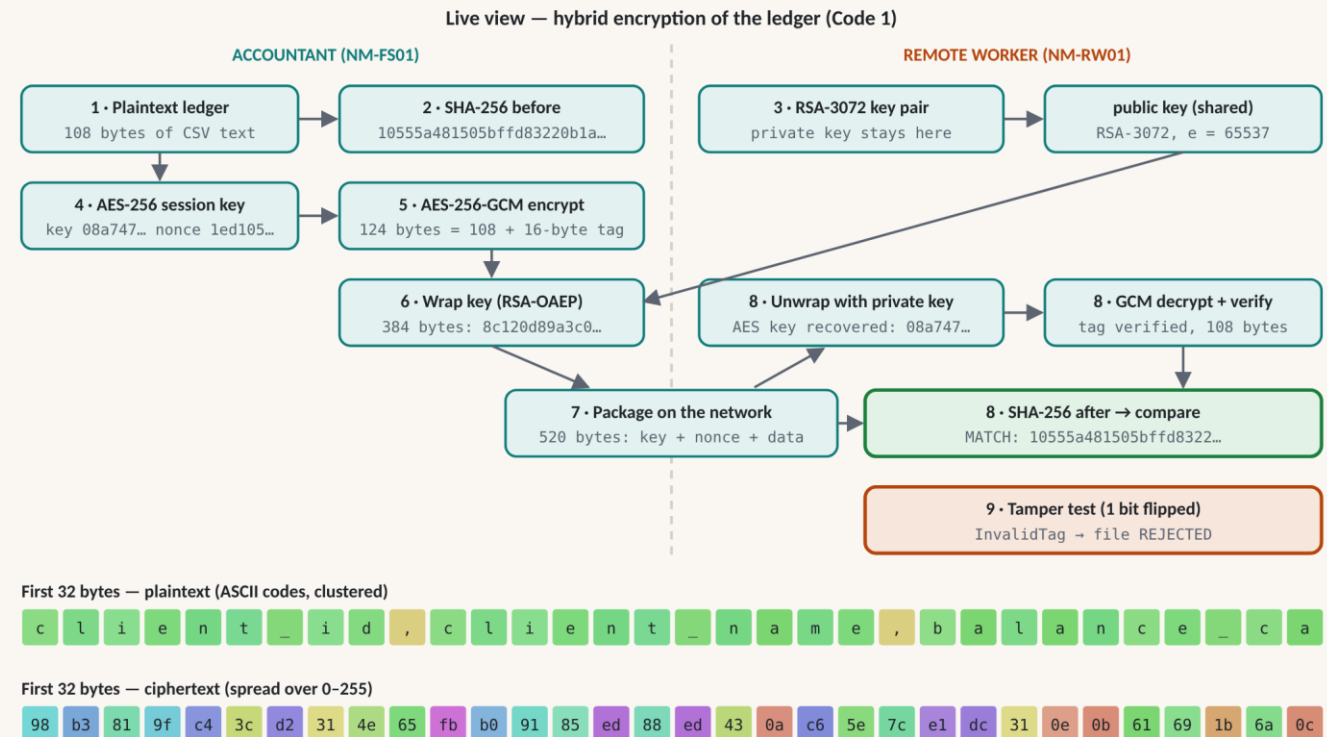
A university lecturer explains why a public key can encrypt but not decrypt. Optional second view.

Third-party videos: linked, not endorsed; check every link with the VirusTotal URL scanner before opening it (Legal Notice, sections 10 and 12).



Real-life example 1: sending the Q3 ledger to the remote worker

- **Task:** only the remote worker can read the ledger; the key never travels in clear; prove the file arrived unchanged
- **SHA-256** (Secure Hash Algorithm, 256-bit output) before → **AES-256-GCM** (Advanced Encryption Standard, 256-bit key, Galois/Counter Mode) encrypt → wrap the **AES** (Advanced Encryption Standard) key with her **RSA-3072** (Rivest–Shamir–Adleman, 3072-bit modulus) public key → send
- NM-RW01: unwrap with the private key → decrypt → **SHA-256** after = **MATCH**
- Tamper test: 1 flipped bit → **InvalidTag** → file **rejected**
- Code 1: `c14_L1_hybrid_encryption.py` (Python 3 + cryptography)
- Returns in: guided exercise · LAB_06 part 1 (WireGuard key pair) · LAB_06 part 2 (encrypted data)



2

LESSON 2 OF 4 · ABOUT 13 MINUTES

Secure storage of credentials

Why systems store hashes, and what salting means

- 2.1 Verify a password without keeping it
- 2.2 Hash-based verification
- 2.3 Why a fast, unsalted hash fails
- 2.4 Salting
- 2.5 Pepper and work factor
- 2.6 Where systems keep hashes
- 2.7 Verifying safely and choosing passwords



The problem: verify a password without keeping it

- A password is an **authentication factor**: something you know. The system must keep some reference value to check it

✗ Plaintext

Anyone who reads the database reads every password.
rockyou.txt: about 32 million plaintext passwords (2009)

✗ Reversible encryption

The decryption key sits on the same server: steal both and read everything

✓ One-way hash

Store only a hash; the password itself is never kept



How hash-based verification works

- **Enrollment**: hash the new password and store the record · **Verification**: hash the typed password the same way and compare
- Equal values = correct password. A stolen table gives digests, not passwords

Figure 2.1 — Enrollment and verification of a password with a salted, slow hash

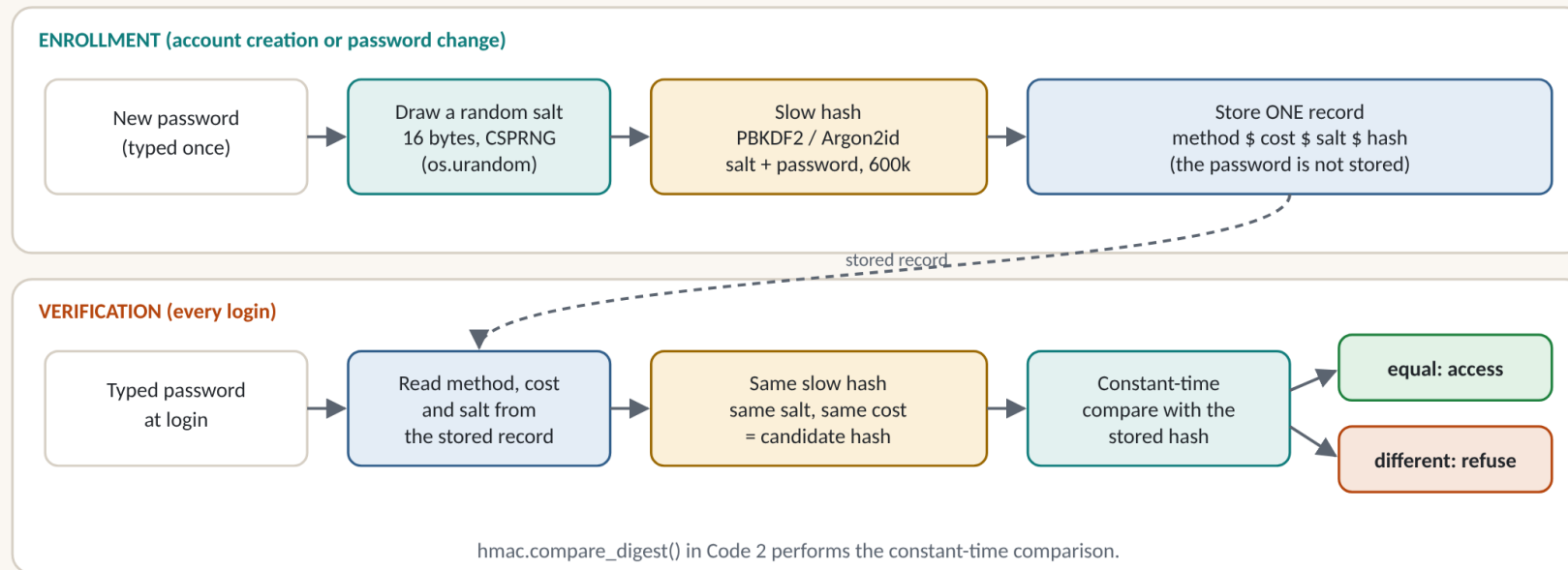


Figure 2.1: enrollment and verification with a salted, slow hash



Why a fast, unsalted hash is not enough

- **Identical passwords → identical digests**: reuse is visible to anyone who reads the table
- **Lookup table**: precomputed pairs digest → password
- **Rainbow table**: compressed chains of hash and reduction steps; trades computing time for storage
- Both are prepared once and reused against every system with the same unsalted hash
- **Fast by design**: under 1 μ s per **SHA-256** (Secure Hash Algorithm, 256-bit output) on one core; billions per second on a **GPU** (Graphics Processing Unit)
- Windows **NT (New Technology) hash** = **MD4** (Message Digest 4) of the password, **not salted** (Microsoft) → protect the **SAM** (Security Account Manager) and Active Directory files

Sources and links: <https://learn.microsoft.com/en-us/windows-server/security/kerberos/passwords-technical-overview>

3 of 4

lab passwords recovered instantly
by a table lookup in Code 2



Salting

- **Salt**: a unique random value per password, combined with it before hashing, stored in clear next to the hash
- **NIST** (National Institute of Standards and Technology): at least 32 bits; common choice 128 bits (16 bytes)
- Effect 1: identical passwords → different hashes
- Effect 2: precomputed tables become useless
- Effect 3: each account must be attacked separately
- A salt is **not secret** and does not make a weak password strong

SAME PASSWORD, TWO SALTS (RECORDED RUN OF CODE 2)

```
accountant1  Maple2026!  
  salt  14jv57seoQ6SLYbIom8ZFQ==  
  hash  W/eEUdBJo68M/+E2wvGppr...
```

```
accountant2  Maple2026!  
  salt  yWxLOpmr1DkBA3zvgFiW4A==  
  hash  3SEs1JxQpix/1qAP15+QpB...
```

Sources and links: <https://pages.nist.gov/800-63-4/sp800-63b.html>



Pepper and work factor (key stretching)

- **Pepper**: one secret value added to all passwords, stored outside the database (configuration, **HSM** (Hardware Security Module))
- **Key stretching**: a deliberately slow, tunable hash · **Work factor**: the tunable cost
- One login pays about 0.2 s; an attacker pays it for every guess
- **Memory-hard**: needs a large amount of **RAM** (Random-Access Memory) per guess, which limits **GPU** (Graphics Processing Unit) cracking

Function	Recommended setting (OWASP)
Argon2id (memory-hard)	m = 19 MiB (mebibytes, 2^{20} bytes each), t = 2, p = 1
scrypt (memory-hard)	N = 2^{17} (128 MiB), r = 8, p = 1
bcrypt	work factor 10 or more; reads only 72 bytes
PBKDF2-HMAC-SHA256 (Password-Based Key Derivation Function 2 using HMAC with SHA-256)	600,000 iterations
yescrypt (memory-hard)	default for /etc/shadow on Debian, Ubuntu, Fedora

Sources and links: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html · <https://www.rfc-editor.org/rfc/rfc9106>



Where systems keep password hashes (1/2)

- Linux: names in `/etc/passwd` (readable by all) · hashes in `/etc/shadow` (root only)
- Hash field: `$id$salt$hash` (yescrypt adds a parameter field)
- Windows: **SAM (Security Account Manager)** (local accounts) and the **Active Directory** database store the unsalted **NT (New Technology)** hash: protect them

Prefix	Method	Status in crypt(5)
\$y\$	yescrypt	Recommended for new hashes
\$gy\$	gost-yescrypt	Recommended for new hashes
\$7\$	scrypt	Listed
\$2b\$	bcrypt	Listed
\$6\$	sha512crypt (5,000 rounds)	Acceptable for new hashes
\$5\$	sha256crypt	Acceptable for new hashes
\$1\$	md5crypt	Obsolete

Sources and links: <https://manpages.ubuntu.com/manpages/noble/man5/crypt.5.html>



Where systems keep password hashes (2/2): reading a record

Figure 2.2 — Anatomy of two stored credential records

A. One line of /etc/shadow on NM-LX01 (yescrypt, real hash of the lab password Maple2026!)

`accountant1:$y$j9T$Qx7pN2vLm4Rt8KsW1aZcE0$kreSSjE5cQLqXYzaPM5l1jd/9cBa1g06tuSWupE5c35:20726:0:99999:7:::`

user name cost parameters salt (stored in clear) hash min / max / warn / inactive / expire

method: yescrypt last change (days since 1970-01-01)

B. One PBKDF2 record written by Code 2 (algorithm \$ iterations \$ salt \$ hash, Base64)

`pbkdf2_sha256$600000$14jv57seoQ6SLYbIom8ZFQ==$W/eUdBJo68M/+E2wvGpprkFs6jq6sIZ2h80raD5+80=`

algorithm work factor: 600,000 iterations 16-byte random salt 32-byte derived hash

Read the record from left to right: the method tells the system which function to run, the parameters say how expensive it is, the salt is combined with the typed password, and the result is compared with the stored hash. Nothing in the record can be decrypted: to test a guess, an attacker must pay the full cost of the function for every guess and every salt.

- On your LAB_02 machine NM-LX01: `sudo cat /etc/shadow` and read the `$y$` records. Reading the format is the exercise; cracking is not part of this course



Verifying safely and choosing good passwords

- **Constant-time comparison**: always compares every byte, so time reveals nothing (Python `hmac.compare_digest`)
- **NIST** (National Institute of Standards and Technology) **SP** (Special Publication) 800-63B-4 (2025): at least **15 characters** when the password is the only factor; at least 8 with **MFA** (Multi-Factor Authentication)
- Accept at least 64 characters · no composition rules · no forced periodic changes (unless compromised)
- Check new passwords against a **blocklist** of common and breached passwords
- Pwned Passwords: only 5 characters of the **SHA-1** (Secure Hash Algorithm 1) hash leave your computer
- **MFA** makes a stolen password insufficient; a password manager makes long, unique passwords practical

15+

characters for a single-factor password (**NIST**, 2025)

Sources and links: <https://csrc.nist.gov/pubs/sp/800/63/b/4/final> · <https://haveibeenpwned.com/Passwords> · <https://docs.python.org/3/library/hmac.html>



Videos for Lesson 2

LESSON 2 · PASSWORD STORAGE



How NOT to Store Passwords! - Computerphile

Computerphile · 9:24 · <https://www.youtube.com/watch?v=8ZtlnClXe1Q>

Plaintext, encryption, plain hashing, salting and slow hashing, in the same order as this lesson.



Hashing and Digital Signatures - CompTIA Security+ SY0-701 - 1.4

Professor Messer · 10:24 · <https://www.youtube.com/watch?v=EcGmQjl6XEO>

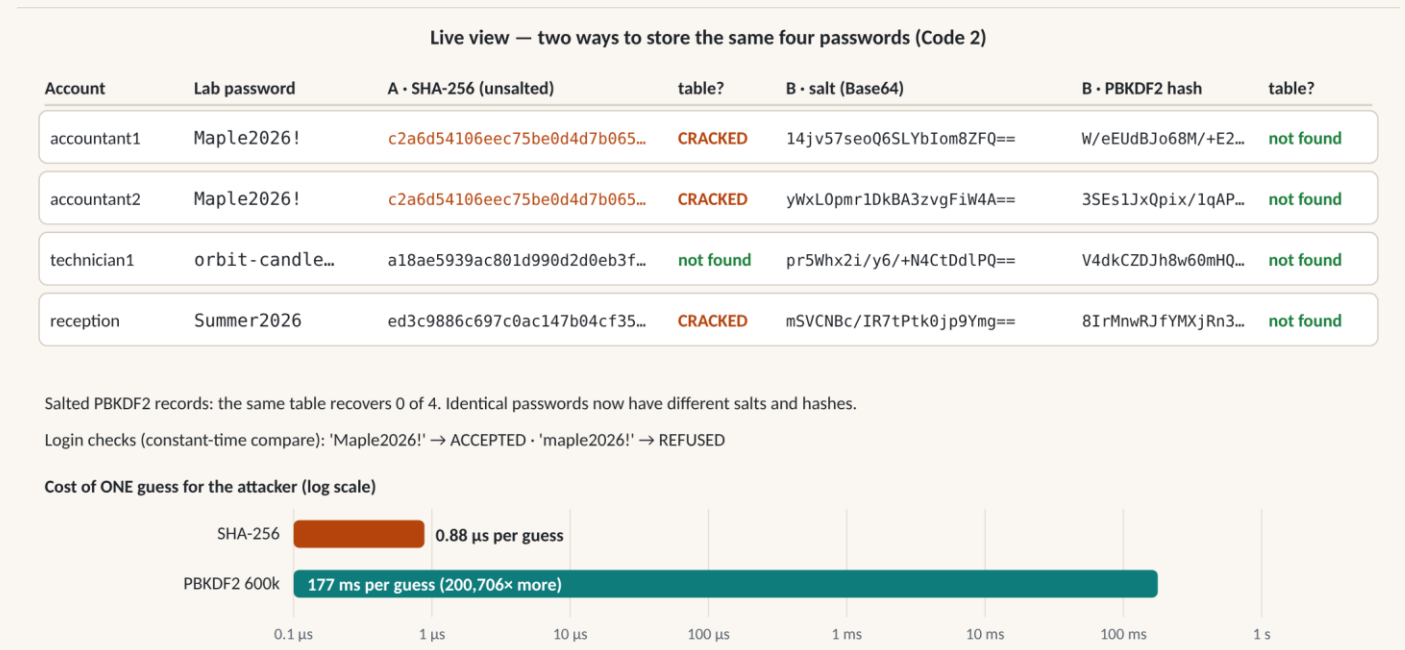
Second half of the video: password storage with hashing, salting and rainbow tables.

Third-party videos: linked, not endorsed; check every link with the VirusTotal URL scanner before opening it (Legal Notice, sections 10 and 12).



Real-life example 2: how NorthMaple stores staff passwords

- **Task:** choose and prove the storage method before any service gets a login page
- A · plain **SHA-256** (Secure Hash Algorithm, 256-bit output): identical digests for the two accountants; **3 of 4** cracked by a table
- B · **PBKDF2-HMAC-SHA256** (Password-Based Key Derivation Function 2 using HMAC with SHA-256), 16-byte salt, 600,000 iterations: **0 of 4**
- Login check: `Maple2026!` accepted, `maple2026!` refused
- Cost per guess: 0.88 μ s vs 177 ms, about **200,000 $\times$** more
- Returns in: guided exercise (`openssl passwd -6`) · LAB_02 hashes · Final Project rows 2 and 9



3

LESSON 3 OF 4 · ABOUT 13 MINUTES

Where cryptography protects data

Data in transit and data at rest

- 3.1 The three states of data
- 3.2 Protecting data in transit
- 3.3 Protecting data at rest
- 3.4 Key management
- 3.5 Why the law cares



The three states of data

At rest

Stored: disks, **SSDs** (Solid-State Drives), **USB** (Universal Serial Bus) drives, backups, databases, cloud storage

In transit

Moving on a network: **LAN** (Local Area Network), Wi-Fi, Internet, **VPN** (Virtual Private Network) tunnel

In use

Being processed in **RAM** (Random-Access Memory) by the **CPU** (Central Processing Unit): protected by operating-system access control

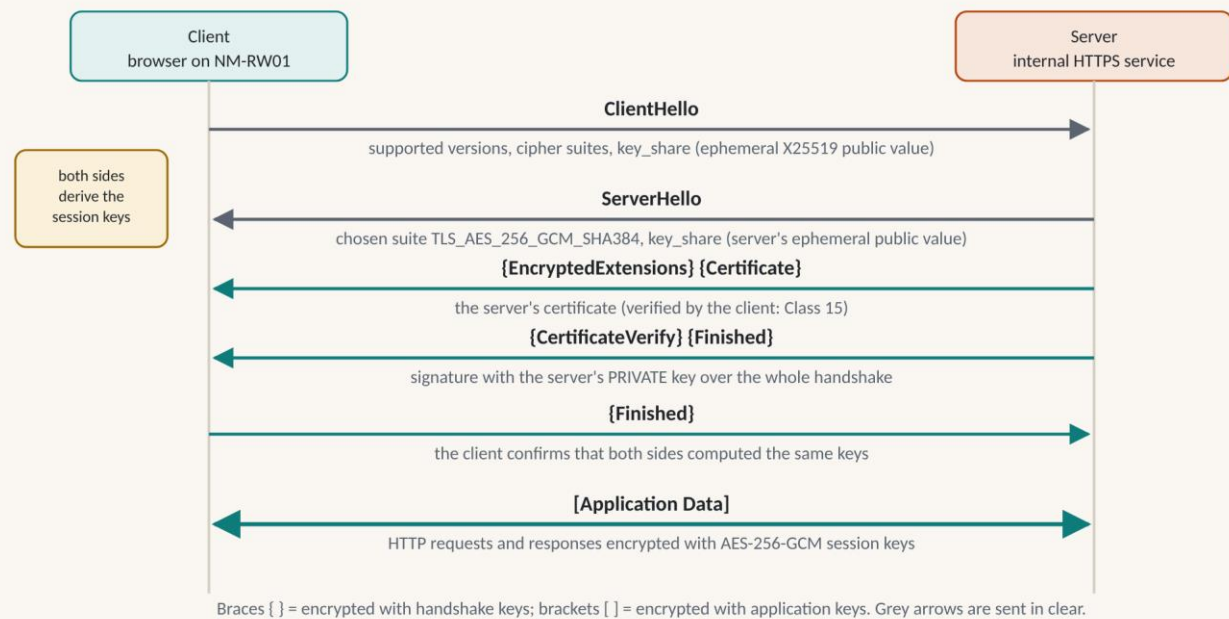
- One file, three states: the ledger on NM-FS01 (at rest) → WireGuard to NM-RW01 (in transit) → opened in a spreadsheet (in use)
- A control that protects one state does nothing for the others



Data in transit (1/2): TLS (Transport Layer Security) 1.3

- Threats: **eavesdropping** (T1040), modification, impersonation
- **TLS 1.3** (RFC (Request for Comments) 8446, 2018) in two phases:
 - **Handshake**: versions, cipher suites, ephemeral **ECDHE** (Elliptic-Curve Diffie–Hellman Ephemeral) key shares, certificate, signature → session keys
 - **Record protocol**: data encrypted with **AEAD** (Authenticated Encryption with Associated Data)
- **Forward secrecy**: a stolen server key cannot decrypt recorded past sessions
- **HTTPS** (Hypertext Transfer Protocol Secure) = **HTTP** (Hypertext Transfer Protocol) inside **TLS** (TCP (Transmission Control Protocol) 443) · **TLS 1.0** and **1.1** deprecated (**RFC 8996**)

Figure 3.1 — The TLS 1.3 handshake (one round trip), then encrypted application data



Sources and links: <https://www.rfc-editor.org/rfc/rfc8446> · <https://www.rfc-editor.org/rfc/rfc8996>



Data in transit (2/2): other protocols and proof

- **SSH (Secure Shell)** (port 22): administration, **SFTP/SCP** (SFTP: SSH File Transfer Protocol; SCP: Secure Copy Protocol) · **IPsec (Internet Protocol Security)** (**ESP** (Encapsulating Security Payload), **IKE** (Internet Key Exchange)): site-to-site **VPNs** (Virtual Private Networks)
- **WireGuard**: Curve25519 keys, ChaCha20-Poly1305, counter-based replay protection (LAB_06)
- **WPA3 (Wi-Fi Protected Access 3)**: protects the radio link only
- **Link encryption** (hop by hop) vs **end-to-end encryption** (sender to final receiver)
- WireGuard NM-RW01 → NM-FW01: tunnel encryption across café Wi-Fi and the Internet
- Prove it: Wireshark shows only **TLS** (Transport Layer Security) Application Data; `openssl s_client` shows the version

RECORDED RUN OF CODE 3, PART C

```
$ openssl s_client -connect 127.0.0.1:8443 \
    -tls1_3 -brief
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384

$ ... -tls1_2 ...
[C2] TLS 1.2 REFUSED
```

Sources and links: <https://www.wireguard.com/protocol/> · <https://www.wireshark.org/>



Data at rest (1/2): full-disk encryption

- Threats: lost or stolen laptop (resilience row 5), discarded disks, stolen backups, open cloud buckets
- **Full-disk encryption**: the whole volume; transparent after unlock
- **BitLocker**: **XTS-AES** (Advanced Encryption Standard in XEX-based Tweaked-codebook mode with ciphertext Stealing) 128 by default, **TPM** (Trusted Platform Module) checks boot integrity · **LUKS (Linux Unified Key Setup)** (cryptsetup) · **FileVault**
- Limit: protects a device that is **off, hibernated or at the pre-boot unlock screen**
- It does **not** protect a running system, even screen-locked, nor stop ransomware running as the user

OFF

Disk = ciphertext without the key

RUNNING

The OS decrypts for every process, malware included

Sources and links: <https://learn.microsoft.com/en-us/windows/security/operating-system-security/data-protection/bitlocker/>



Data at rest (2/2): files, databases and cloud

- **File and container encryption:** **EFS** (Encrypting File System), VeraCrypt, 7-Zip **AES-256** (Advanced Encryption Standard, 256-bit key), GnuPG, `openssl enc`
 - Containers and encrypted archives keep their protection when copied; **EFS** only while on **NTFS** (New Technology File System)
 - `openssl enc` gives confidentiality only: it does not support **GCM** (Galois/Counter Mode)
- **Databases:** **TDE** (Transparent Data Encryption) encrypts the database files; column-level encryption for specific fields
- **Cloud:** Amazon **S3** (Simple Storage Service) encrypts every new object since January 5, 2023 (**SSE-S3** (Server-Side Encryption with Amazon S3 managed keys), **AES-256**)
 - Azure Storage encryption is always on (**AES-256**), Microsoft- or customer-managed keys
- Shared responsibility: provider encryption does not protect a bucket you made public

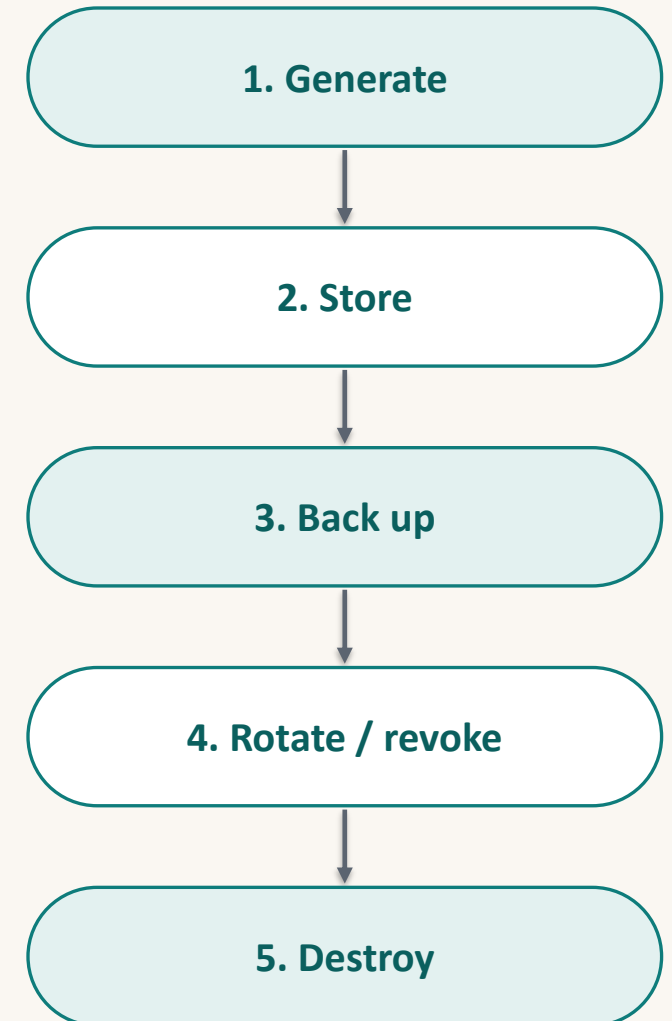


Sources and links: <https://veracrypt.io/> · <https://docs.openssl.org/master/man1/openssl-enc/> · <https://docs.aws.amazon.com/AmazonS3/latest/userguide/default-encryption-faq.html> · <https://learn.microsoft.com/en-us/azure/storage/common/storage-service-encryption>



Key management

- Once data is encrypted, its security depends on the protection of the keys
- **Generate** keys with a **CSPRNG (Cryptographically Secure Pseudo-Random Number Generator)**, never from a guessable value
- **Store** keys apart from the data: **TPM** (Trusted Platform Module), **HSM** (Hardware Security Module), cloud **KMS** (Key Management Service), file mode 600
- **Back up** recovery keys (BitLocker: a 48-digit number)
- **Rotate and revoke**: remove a lost device's WireGuard peer at once
- **Destroy** every copy of a key = **crypto-erase**



Why the law cares

- NorthMaple holds client financial records: **personal information**
- **PIPEDA** (Personal Information Protection and Electronic Documents Act), Schedule 1, 4.7.1: safeguards appropriate to the sensitivity of the information
- 4.7.3 (c): "technological measures, for example, the use of passwords and encryption"
- Québec Law 25: notify the **CAI (Commission d'accès à l'information du Québec)** and the persons concerned promptly when a **confidentiality incident** presents a risk of serious injury; keep an incident register
- A lost laptop that is fully encrypted, key not exposed: misuse is much less likely
- Educational context, not legal advice (Legal Notice, section 7)



Sources and links: <https://laws-lois.justice.gc.ca/eng/acts/P-8.6/FullText.html> · <https://www.cai.gouv.qc.ca/protection-renseignements-personnels/information-entreprises-privées/incidents-confidentialite-mesures-securite-entreprises>



Videos for Lesson 3

LESSON 3 · IN TRANSIT & AT REST



Encrypting Data - CompTIA Security+ SY0-701 - 1.4

Professor Messer · 9:48 · <https://www.youtube.com/watch?v=jpsc4c7Intw>

Full-disk, volume, file, database and column encryption for data at rest, then transport encryption (**HTTPS** (Hypertext Transfer Protocol Secure), **VPN** (Virtual Private Network), **TLS** (Transport Layer Security), **IPsec** (Internet Protocol Security)) and key length.



States of Data - CompTIA Security+ SY0-701 - 3.3

Professor Messer · 6:07 · <https://www.youtube.com/watch?v=71RQaYQ4QSw>

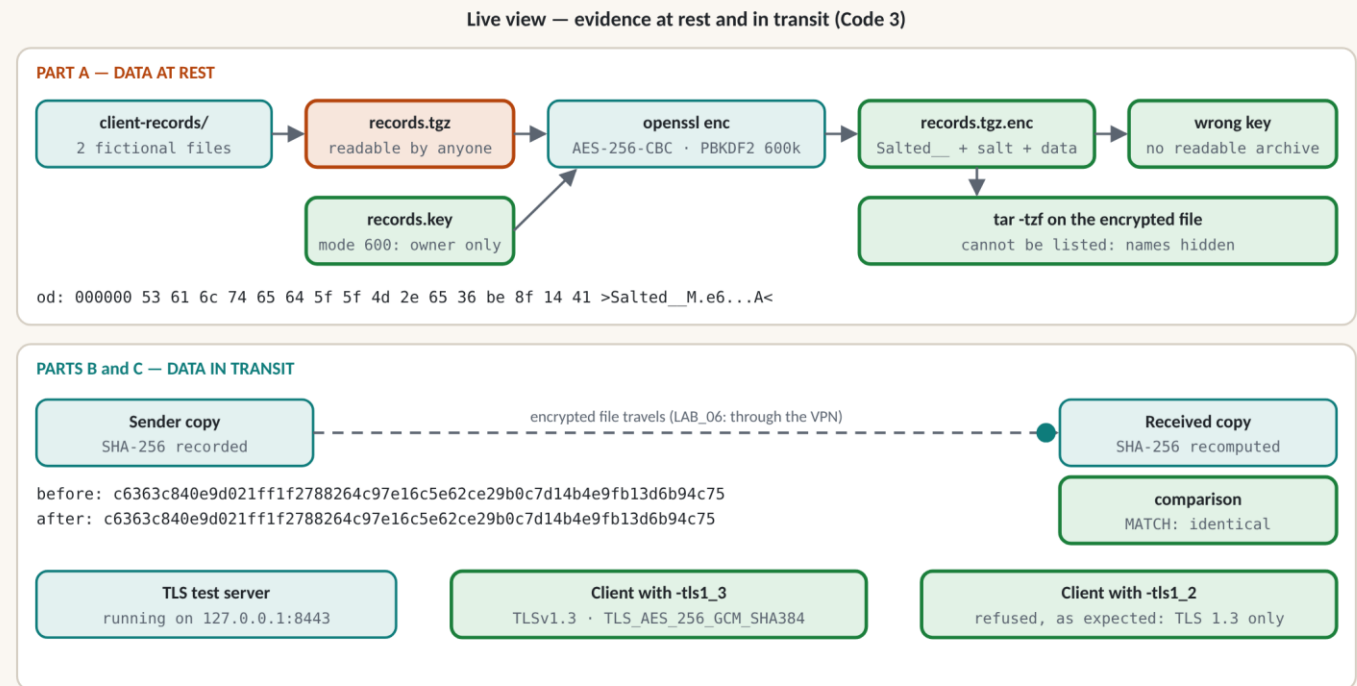
Data at rest, in transit and in use, with the controls for each state.

Third-party videos: linked, not endorsed; check every link with the VirusTotal URL scanner before opening it (Legal Notice, sections 10 and 12).



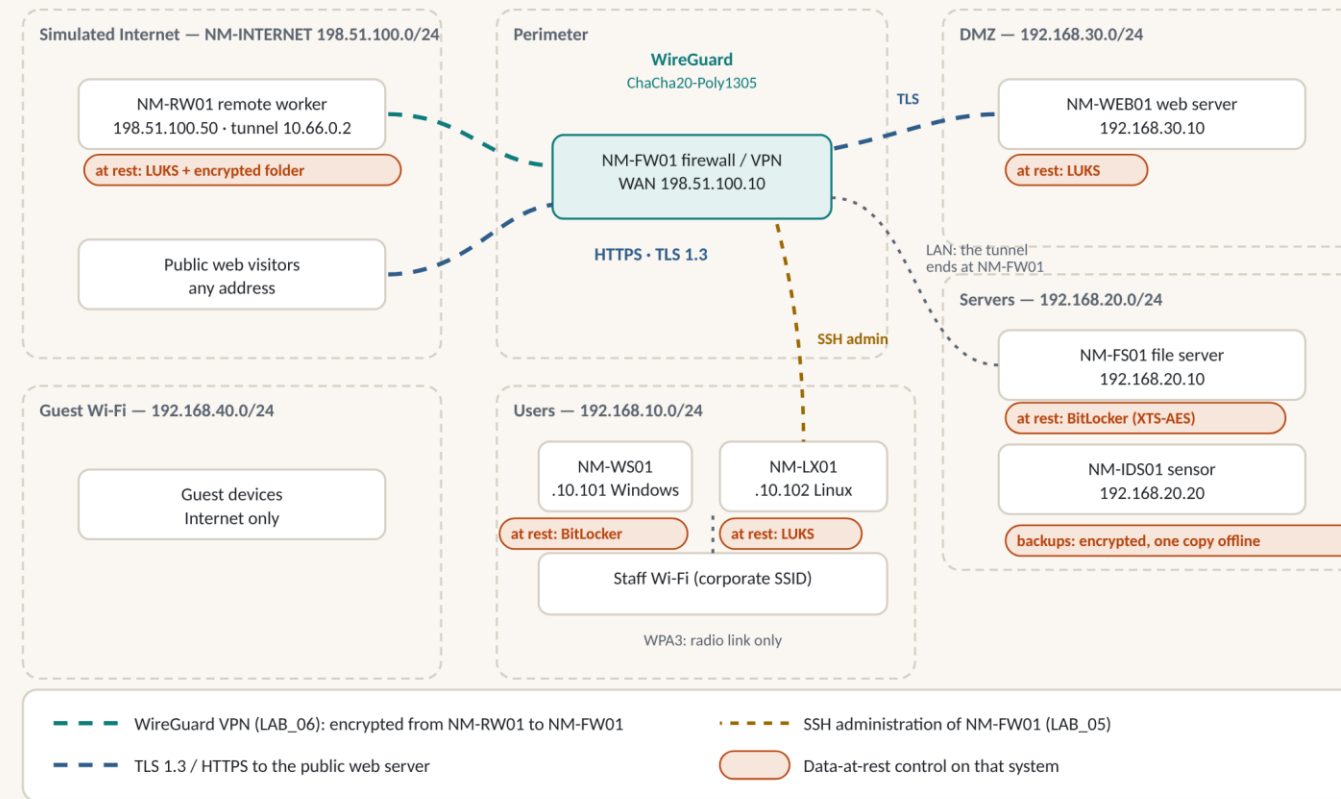
Real-life example 3: proving protection at rest and in transit

- **Task:** show the owner three pieces of evidence, not a promise
- A · at rest: the encrypted archive cannot be listed; a wrong key gives no readable data
- B · in transit: **SHA-256** (Secure Hash Algorithm, 256-bit output) before = after → **MATCH**
- C · the service accepts **TLS (Transport Layer Security) 1.3** and refuses **TLS 1.2**
- Code 3:
`c14_L3_rest_transit_evidence.sh`
(Bash + OpenSSL, Ubuntu)
- Returns in: LAB_06 part 2 validation · rows 5 and 6 of the resilience assessment



NorthMaple: where cryptography protects data

Figure 3.2 — NorthMaple Accounting Inc.: where cryptography protects data (addresses from the course VM plan)



- Encrypted paths: WireGuard (NM-RW01 → NM-FW01), **HTTPS** (Hypertext Transfer Protocol Secure) to NM-WEB01, **SSH** (Secure Shell) administration · rust tags = data-at-rest controls



4

LESSON 4 OF 4 · ABOUT 11 MINUTES

Attacks on cryptosystems and simple defenses

Awareness level

- 4.1 Attacks on keys and passwords
- 4.2 Attacks on the communication
- 4.3 Attacks on algorithms, and quantum computing
- 4.4 Attacks on the implementation
- 4.5 Key theft, people and ransomware



Attacks on keys and passwords (1/2): exhaustive search

- **Cryptanalysis**: the study of methods for defeating cryptographic protection
- **Brute-force attack**: try every key; average time = $(\text{alphabet}^{\text{length}} \div 2) \div \text{guesses per second}$
- Random **AES-128** (Advanced Encryption Standard, 128-bit key) key at 10^{12} keys/s: about 5.4×10^{18} years (the universe: 1.4×10^{10} years)
- Exhaustive search works only against short or human-chosen passwords
- **Dictionary attack**: words from leaked-password lists (rockyou.txt)
- **Rule-based attack**: predictable changes: Summer2026, Maple2026!

5.4×10^{18}

years to exhaust half of the **AES-128** key space at 10^{12} keys/s



Attacks on keys and passwords (2/2): online vs offline

- Precomputed tables (defeated by salts) · **Password spraying** · **Credential stuffing** (MITRE **ATT&CK** (Adversarial Tactics, Techniques, and Common Knowledge) T1110)
- **Online attack**: through the live login → rate limiting, lockout, **MFA** (Multi-Factor Authentication), **IDS** (Intrusion Detection System) alerts · **Offline attack**: stolen hashes → only password strength and work factor

Figure 4.2 — Online and offline password attacks: two paths, two sets of defenses

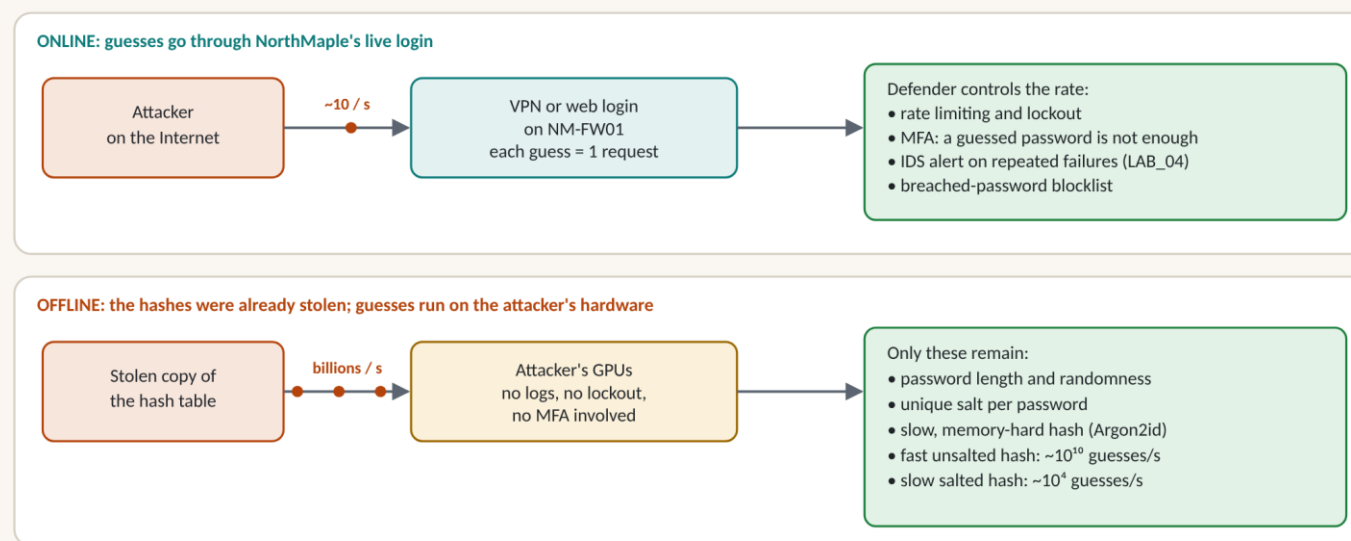


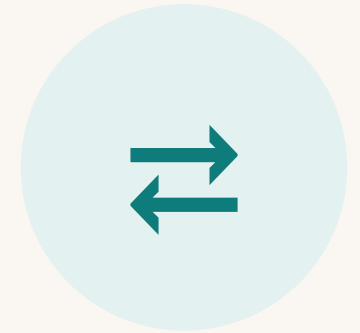
Figure 4.2: online and offline password attacks

Sources and links: <https://attack.mitre.org/techniques/T1110/>



Attacks on the communication

- **On-path attack** (man-in-the-middle): **ARP** (Address Resolution Protocol) poisoning, **DHCP** (Dynamic Host Configuration Protocol) spoofing, **evil twin** (MITRE **ATT&CK** (Adversarial Tactics, Techniques, and Common Knowledge) T1557)
 - Defense: authenticated key exchange: validate certificates, never accept a certificate warning, **VPN** (Virtual Private Network), **WPA3** (Wi-Fi Protected Access 3)
 - **HSTS (HTTP Strict Transport Security)** (RFC (Request for Comments) 6797) forces **HTTPS** (Hypertext Transfer Protocol Secure) and blocks **SSL** (Secure Sockets Layer) stripping after the first visit, or from the first visit for preloaded sites
- **Downgrade attack**: force an old version or weak cipher; **POODLE** (Padding Oracle On Downgraded Legacy Encryption) on **SSL 3.0 (CVE-2014-3566** (Common Vulnerabilities and Exposures))
 - Defense: disable **SSL**, **TLS** (Transport Layer Security) 1.0 and **TLS 1.1**; **TLS 1.3** has a built-in downgrade signal
- **Replay attack**: resend a captured valid message
 - Defense: nonces, timestamps, counters (WireGuard rejects old counters)

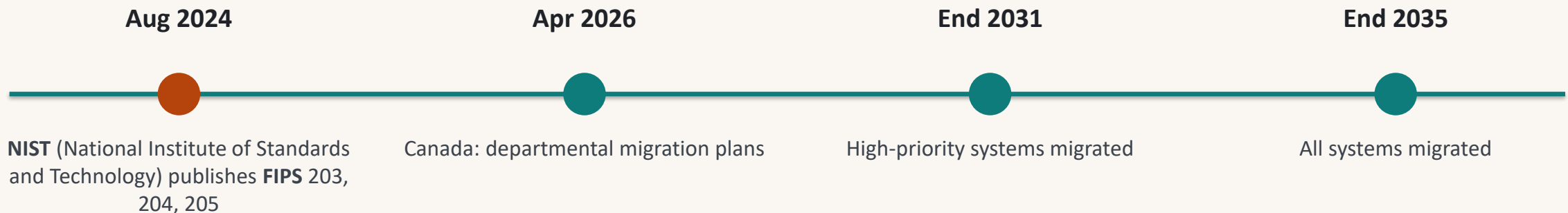


Sources and links: <https://attack.mitre.org/techniques/T1557/> · <https://www.rfc-editor.org/rfc/rfc6797> · <https://nvd.nist.gov/vuln/detail/CVE-2014-3566>



Attacks on algorithms, and the quantum question

- **Collision attack / birthday attack**: two inputs, same hash; about $2^{(n/2)}$ attempts
- **MD5** (Message Digest 5): the Flame malware carried a forged code-signing certificate (2012) · **SHA-1** (Secure Hash Algorithm 1): SHAttered (2017) → use **SHA-256** (Secure Hash Algorithm, 256-bit output) or **SHA-3** (Secure Hash Algorithm 3)
- Quantum: Shor's algorithm would break **RSA** (Rivest–Shamir–Adleman) and **ECC** (Elliptic Curve Cryptography); Grover only halves **AES** (Advanced Encryption Standard) strength (**AES-256** (Advanced Encryption Standard, 256-bit key) stays strong)
- **Harvest now, decrypt later** → **post-quantum cryptography**: **FIPS** (Federal Information Processing Standards) 203 (**ML-KEM** (Module-Lattice-Based Key-Encapsulation Mechanism)), 204 (**ML-DSA** (Module-Lattice-Based Digital Signature Algorithm)), 205 (**SLH-DSA** (Stateless Hash-Based Digital Signature Algorithm))
- **Crypto-agility**: replace algorithms without rebuilding systems



Sources and links: <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards> · <https://www.cyber.gc.ca/en/guidance/roadmap-migration-post-quantum-cryptography-government-canada-itsm40001> · <https://learn.microsoft.com/en-us/security-updates/securityadvisories/2012/2718704>



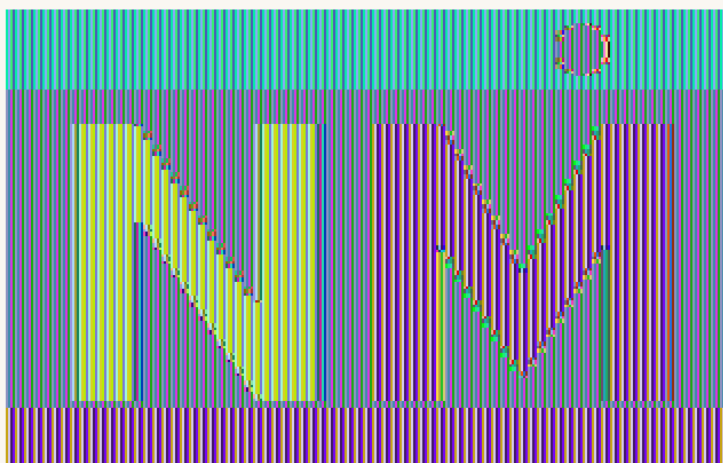
Attacks on the implementation

- **Side-channel attack**: time, power, emissions, cache → constant-time code · Mode misuse: **ECB** (Electronic Codebook) patterns, **GCM** (Galois/Counter Mode) nonce reuse · Weak random numbers → OS **CSPRNG** (Cryptographically Secure Pseudo-Random Number Generator)
- Library flaws: Heartbleed (**CVE-2014-0160** (Common Vulnerabilities and Exposures)) → patching · Hard-coded keys → secrets manager · Never design your own crypto (**OWASP** (Open Worldwide Application Security Project) A04:2025)

Original image (plaintext)



AES-128-ECB (pattern leaks)



AES-128-GCM (no pattern)



Figure 4.1: the same image encrypted with **AES-128-ECB** (pattern visible) and **AES-128-GCM** (no pattern)

Sources and links: <https://nvd.nist.gov/vuln/detail/CVE-2014-0160> · https://top10.owasp.org/2025/A04_2025-Cryptographic_Failures/



Key theft, people, and ransomware

- The easiest way to decrypt is to **obtain the key**: a copied private key, a recovery key in a shared folder, **VPN** (Virtual Private Network) credentials typed on a phishing page
- **Ransomware**: attackers encrypt your data and demand payment (MITRE **ATT&CK** (Adversarial Tactics, Techniques, and Common Knowledge) T1486); your own encryption does not prevent it

Protect keys

File mode 600, **TPM** (Trusted Platform Module) or **HSM** (Hardware Security Module)

MFA (Multi-Factor Authentication) and revocation

Revoke lost devices at once

Offline backups

Immutable or offline copy (row 10)

Layers

Least privilege (LAB_02), detection (LAB_04), segmentation (LAB_05)

Sources and links: <https://attack.mitre.org/techniques/T1486/>



Attack-to-defense matrix

Attack	Simple defense	MITRE ATT&CK
Brute force	Long random secrets; key-based authentication	T1110
Dictionary / rule-based	Breached-password blocklist, 15+ characters, MFA (Multi-Factor Authentication)	T1110.001/.002
Lookup / rainbow tables	Unique salt per password	T1110.002
Password spraying	MFA , smart lockout, monitoring	T1110.003
Credential stuffing	Unique passwords, MFA	T1110.004
On-path	Certificate validation, HSTS (HTTP Strict Transport Security), VPN (Virtual Private Network), WPA3 (Wi-Fi Protected Access 3)	T1557
Downgrade	Disable SSL (Secure Sockets Layer), TLS (Transport Layer Security) 1.0, TLS 1.1	—
Replay	Nonces, timestamps, counters	—
Collision	SHA-256 (Secure Hash Algorithm, 256-bit output) or SHA-3 (Secure Hash Algorithm 3)	—
Side channel	Constant-time code	—
Implementation flaw	Patch management	—
Key theft / phishing	Key protection, MFA , revocation, training	T1552.004, T1566
Harvest now, decrypt later	Post-quantum algorithms, crypto-agility	—
Ransomware	Offline backups, least privilege, detection	T1486



Videos for Lesson 4

LESSON 4 · ATTACKS & DEFENSES



Cryptographic Attacks - CompTIA Security+ SY0-701 - 2.4

Professor Messer · 9:31 ·

<https://www.youtube.com/watch?v=7aJaEQy6Yoc>

Birthday attacks and hash collisions, **MD5** (Message Digest 5), downgrade attacks and **SSL** (Secure Sockets Layer) stripping.



Password Attacks - CompTIA Security+ SY0-701 - 2.4

Professor Messer · 7:15 · <https://www.youtube.com/watch?v=-ZfbifHwEVE>

Plaintext storage risks, hashes, account lockout, password spraying and offline brute force.



On-path Attacks - CompTIA Security+ SY0-701 - 2.4

Professor Messer · 5:30 ·

https://www.youtube.com/watch?v=M_Af6_8JTuo

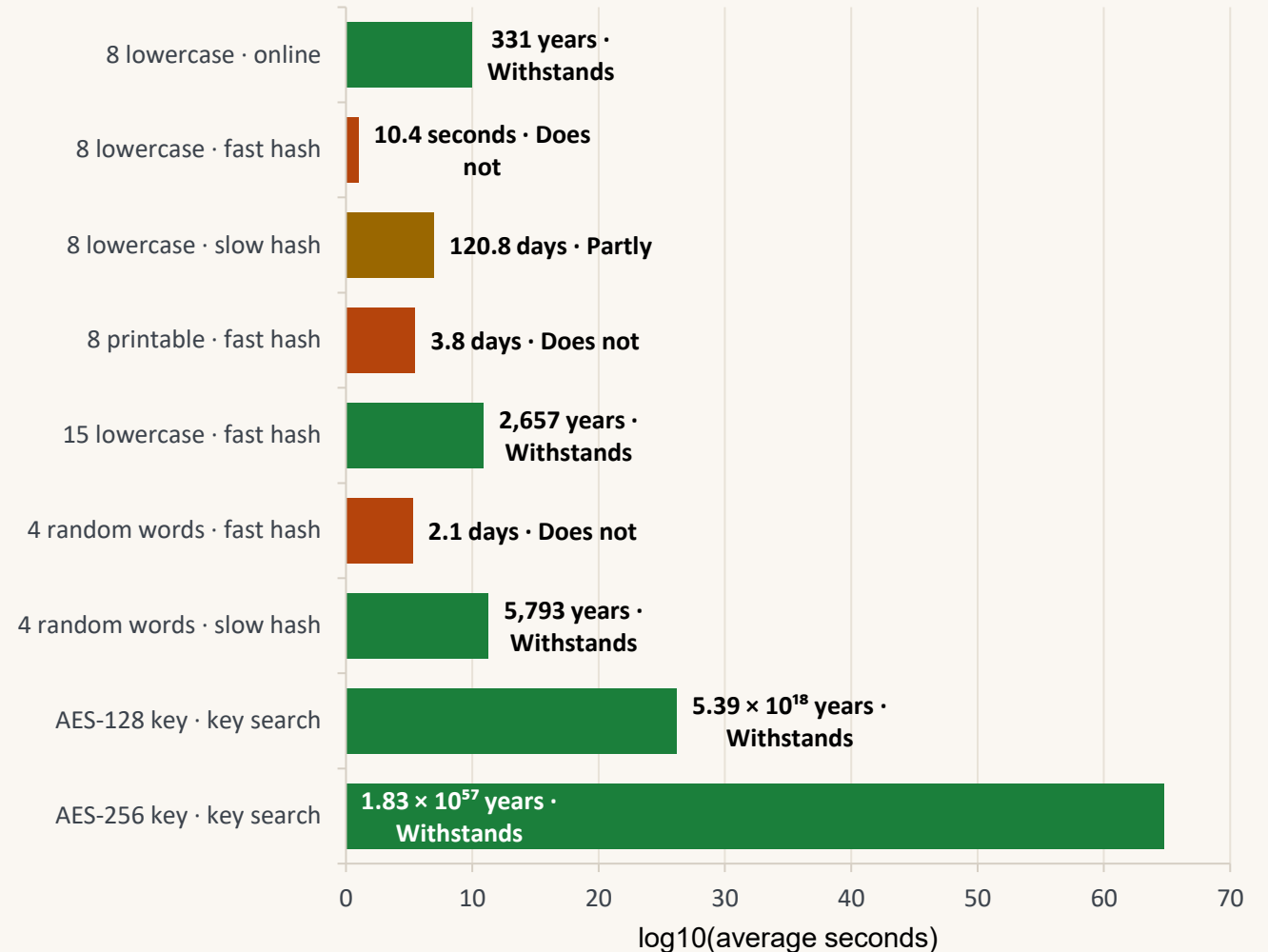
How an attacker gets in the middle of a conversation (**ARP** (Address Resolution Protocol) poisoning, on-path browser attacks).

Third-party videos: linked, not endorsed; check every link with the VirusTotal URL scanner before opening it (Legal Notice, sections 10 and 12).



Real-life example 4: rating NorthMaple's secrets

- **Task:** justify row 2 of the resilience assessment with numbers
- Same 8-letter password: online 331 years (Withstands) · stolen fast hash **10 seconds** (Does not) · slow salted hash 121 days (Partly)
- 15 letters: 2,657 years even with a fast hash · 4 random words: 5,793 years with a slow hash
- Random **AES** (Advanced Encryption Standard) keys: beyond any physical possibility
- Thresholds: ≥ 100 years Withstands · ≥ 30 days Partly
- Code 4:
[c14_L4_bruteforce_estimator.html](#)
(open in a browser)



Class 14 in one table

Concept	What it gives	Where you use it next
Symmetric encryption (AES-GCM (Advanced Encryption Standard in Galois/Counter Mode), ChaCha20)	Confidentiality; integrity with AEAD (Authenticated Encryption with Associated Data)	LAB_06 part 2 (encrypted folder)
Asymmetric cryptography (RSA (Rivest–Shamir–Adleman), ECC (Elliptic Curve Cryptography))	Key exchange, signatures, key transport	LAB_06 part 1 (WireGuard); Class 15
Hash function (SHA-256 (Secure Hash Algorithm, 256-bit output))	Integrity evidence	Guided exercise; Chapter 6 validation
Salted slow password hash	Safe password storage	Resilience rows 2 and 9
Steganography	Hidden data: a detection problem	IDS (Intrusion Detection System) awareness
Data-in-transit controls	Protection of traffic	LAB_06 (VPN (Virtual Private Network), HTTPS (Hypertext Transfer Protocol Secure)); row 6
Data-at-rest controls	Protection of stored data	LAB_06 part 2; row 5
Attack awareness	Choosing the right defense	Resilience assessment (Chapter 7)



Guided exercise after the break (not graded)

- On NM-LX01 (Linux) or NM-WS01 (Windows), in your lab only
- 1. Create `nm_test.txt` · 2. Hash it with OpenSSL
- 3. Change one character, hash again: compare (avalanche)
- 4. `openssl passwd -6` twice: different results (random salt)
- 5. Same command with a fixed salt twice: identical. Why?
- 6. Write the digests and your explanation in your manual draft
- Windows: `Get-FileHash .\nm_test.txt -Algorithm SHA256`

COMMANDS

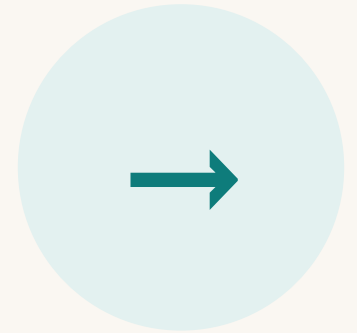
```
$ echo "NorthMaple lab file - Class 14" \  
  > nm_test.txt  
$ openssl dgst -sha256 nm_test.txt  
$ openssl passwd -6 'Lab-Only-Passw0rd'  
$ openssl passwd -6 -salt NMlab2026 \  
  'Lab-Only-Passw0rd'  
$6$NMlab2026$3Fn0xRW8A8ZJ7oe77B5N...
```

Sources and links: <https://docs.openssl.org/master/man1/openssl-passwd/> · <https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash>



LAB_06 at a glance, and next steps

- **LAB_06** (Classes 16 and 17, 5 %): the full lab brief is distributed separately
 - Part 1: remote-access **VPN** (Virtual Private Network) into NorthMaple (WireGuard recommended); connect the remote-worker client
 - Part 2: encrypt one folder or volume; **HTTPS** (Hypertext Transfer Protocol Secure) with a self-signed certificate on one internal service; validate both
 - Deliverable → Chapter 6 of the Final Project and the Course Procedure Manual
- **Class 15** (Tuesday, October 6): **PKI** (Public Key Infrastructure), certificates, how a browser trusts a website
- **Class 16** (Wednesday, October 7): **VPNs**, LAB_06 part 1, and quiz Q_03
- Before Class 15: check that NM-FW01 still starts; watch the videos (about 85 minutes)



Questions?

Thank you. See you on Tuesday, October 6, for Class 15: PKI.



Educational Hub © 2026 | From the Industry to the Classroom — Building the IT's Future, Today and Together !

ALL Rights Reserved

© 2026 Cyber.SoHo. Personal study licence only: no redistribution, re-publication or public posting (Legal Notice, section 8). Practise only in your isolated lab.

